하네스 엔지니어링
Harness Engineering

에이전트에게 자유를 주려면, 먼저 경계를 설계하라

DK
테스트

2026.03.31

INTRODUCTION

통제 불능의 에이전트, 해결책은 무엇인가?

쏟아지는 자율 생성 코드 속에서, 전체 시스템의 무결성을 보장하는 방법

THE SHIFT

모델은상품이다.
하네스가 해자다.

어떤 모델을 쓰느냐보다, 어떤 하네스 위에서 운용하느냐가 시스템의 성능을 결정한다.

01

CHAPTER

하네스 엔지니어링의 탄생

에이전트 기반 자율 개발의 등장과 엔지니어링 패러다임의 진화

DEFINITION

하네스(Harness)란 무엇인가?

- 에이전트 외부 환경 전체를 설계하는 새로운 공학
- 대규모 소프트웨어의 신뢰성 있는 자율 구축 지원
- 2026년 OpenAI 코덱스 팀의 실제 사례로 공식화
- 에이전트를 억압하지 않고 올바른 방향으로 구조적 인도

HISTORY

AI 엔지니어링 패러다임의 진화

2022~

프롬프트 엔지니어링

에이전트의 입력 문구를 다듬는 방식. 확장성의 한계 직면.

2024~

컨텍스트 엔지니어링

RAG, 지식베이스 연동. 행동의 제약이 없어 일관성 부족.

2025

자율 생성 코드의 증명

에이전트가 스스로 유효한 코드를 쓸 수 있음이 대규모로 입증.

2026~

하네스 엔지니어링

에이전트를 둘러싼 물리적/논리적 환경 전체를 설계하는 단계.

PARADIGM SHIFT

세 패러다임의 근본적 차이

프롬프트

포커스

"무엇을 말할 것인가"

작업 대상

입력 텍스트

핵심 도구

프롬프트 템플릿

한계점

비결정론적 결과

컨텍스트

포커스

"무엇을 알게 할 것인가"

작업 대상

참조용 정보

핵심 도구

RAG 체인 및 DB

한계점

일관성 유지의 한계

하네스

포커스

"어디서 움직일 것인가"

작업 대상

시스템 외부 환경 전체

핵심 도구

린터, CI, GC 자동화

한계점

높은 초기 설계 비용

02

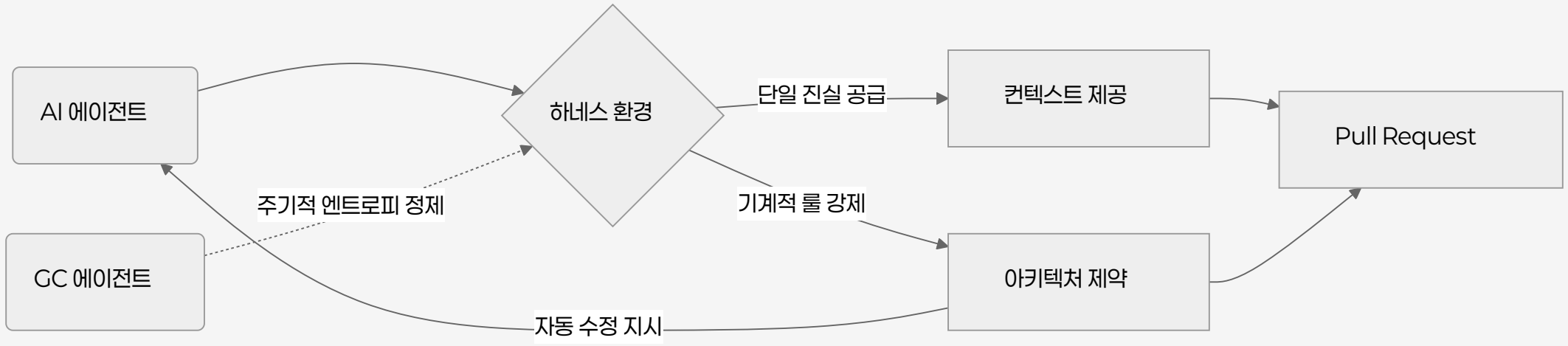
CHAPTER

3대 구성 요소와 작동 원리

컨텍스트, 아키텍처 제약, 가비지 컬렉션의 유기적 결합 메커니즘

SYSTEM ARCHITECTURE

하네스 시스템의 순환 구조



CORE COMPONENTS

하네스의 3대 구성 요소

Context Engineering

에이전트가 참조할 정보 구축.
AGENTS.md, ADR, 실행 계획 관리

Architectural Constraints

아키텍처 경계 이탈을 막는 커스텀 린터 및 구조적 테스트

GC Agents

무질서도(엔트로피)를 자동 스캔하여 코드를 정제하는 전담 레이어

THE PARADOX

제약이 자율성을 확장한다

무한한 자유는 에이전트를 길 잃게 만듭니다. 좁은 해결 공간이 작업의 신뢰성을 보장합니다.

물리적 제약과 자동 피드백 루프

단일 진실 공급원 (SSOT) 구축

외부 문서 없이도 리포지토리 자체가 에이전트의 완결된 지식 기반이 되도록 설계

자동 수정 지시 (Auto-Correction)

린터 에러 발생 시 단순 실패 알림이 아닌, 에이전트용 수정 지시를 자동 주입

주기적 엔트로피 상쇄 루프

에이전트 작업 증가로 인한 코드 붕괴를 주기적으로 실행되는 GC가 스캔하여 방지

ROLE TRANSFORMATION

엔지니어 역할의 재정의

코드 타이피스트

과거의 엔지니어는 언어의 문법을 직접 타이핑하고 발생한 버그를 수동으로 찾아 고치는 데 집중했습니다. 코드 리뷰를 진행하고 아키텍처를 하나하나 직접 구현하는 미시적인 작업에 대부분의 근무 시간을 쏟아야만 했습니다.

환경 설계자

하네스 시대의 엔지니어는 에이전트가 스스로 일할 수 있도록 제약과 피드백 루프를 설계합니다. 아키텍처 규칙을 린터에 인코딩하고, 시스템이 무너지지 않도록 방어선을 쳐 조직 전체의 생산성 저점을 끌어올리는 인프라를 만듭니다.

“

에이전트 처리량이 인간 주의를 훨씬 초과하는 시스템에서,
수정은 싸고 기다림은 비싸다.

OpenAI Codex 팀 (2026)
100만 줄의 프로덕션 코드를 수동 작성 없이 배포하며

03

CHAPTER

실전 구축과 미래 전략

단계별 도입 방법론과 실제 글로벌 기업들의 압도적 성과 및 한계점

IMPLEMENTATION

하네스 구축 3단계 성숙도

1

Level 1

[컨텍스트 구축] AGENTS.md 생성 및 실행 계획 템플릿 표준화

→

2

Level 2

[아키텍처 제약] 커스텀 린터 도입 및 구조적 테스트 CI 연동

→

3

Level 3

[GC 에이전트] 주기적 엔트로피 스캔 및 자가 검증 루프 실행

→

4

최적화

[피드백 강화] 에러 메시지 재설계로 에이전트의 자율 수정 유도

CASE STUDY

글로벌 기업의 하네스 적용 성과

0줄

수동 작성 프로덕션 코드

OpenAI Codex 팀 (5개월)

10배

수동 개발 대비 속도 향상

엔지니어 1인당 하루 3.5개 PR 처리

1,000개+

매주 자동 머지되는 PR

Stripe 6계층 하네스 환경

5회

동일 모델 대상 하네스 재설계

Manus, 신뢰성 대폭 향상 입증

CAUTION

하네스 역시 지속적인 유지보수가 필요한 또 하나의 코드다.

초기 설계 비용이 높고, 비즈니스가 진화함에 따라 아키텍처 규칙도 갱신되어야 한다.

FUTURE OUTLOOK

도입 한계 및 미래 전망

높은 초기 구축 비용 및 오버엔지니어링

5개월의 인프라 투자가 필요했던 OpenAI처럼 소규모 프로젝트엔 과잉 제약 우려

GC 에이전트의 오작동 리스크

유효한 코드가 자동 삭제될 위험을 막기 위해 PR 기반 인간 승인 단계를 유지

벤치마킹 프레임워크의 표준화

AI를 실제 환경 기반으로 묶어두고 평가하는 Harness Bench 등 생태계 확장 예상

Human-Aware SRE로의 확장

개발 영역을 넘어 시스템 안정성과 운영(SRE) 전반으로 하네스 철학이 확대 도입됨

오픈소스 하네스 생태계 형성

팀별 하네스 컴포넌트나 린터 규칙이 마켓플레이스를 통해 공유 및 재사용되는 시대

KEY TAKEAWAYS

환경이 품질을 결정한다

에이전트의 오작동은 능력이 아닌 환경의 문제입니다.

우리 조직만의 고유한 아키텍처 철학을 하네스에 인코딩하십시오.

지금 당장 레포지토리에 AGENTS.md를 추가하는 것으로 시작합시다.

Q & A

PRESENTER DK

SESSION 2026.03.31